

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: Task 5

Student Name: R Varshini

Reg. No.: A24126552115

Date: 21-09-2026

1. TITLE

JavaScript Functions vs Classes – Creating Multiple Objects with Common Properties and Methods using a Student Class

2. AIM

To differentiate between JavaScript functions and classes, and to create a **Student** class in Node.js that is used to build multiple objects sharing common properties (name, age, grade) and a common method (introduce()), and to verify the output.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what a function and a class are in JavaScript.
- To compare functions and classes on syntax, instantiation, hoisting, methods and inheritance.
- To define a class with a constructor and a method.
- To create multiple objects from one class using the **new** keyword.
- To show that every object has its own property values but all objects share the same method.
- To run the program with Node.js and verify the actual output against the expected output.

4. THEORY

- **Function:** a reusable block of code that performs a task when it is called. A function can also act as a constructor (called with **new**) to build objects, but the shared methods must be attached manually to `Function.prototype`.
- **Class:** introduced in ES6, a class is a template (blueprint) for creating objects. It groups the constructor and the methods in one block. Internally a class is still a special kind of function, so it is called "syntactic sugar" over JavaScript's prototype-based inheritance.
- **constructor():** a special method that runs automatically every time an object is created with **new**. It initialises the properties of that object using **this**.
- **Method:** a function written inside the class body. It is stored once on the class prototype and every object created from the class can use it.
- **Object / Instance:** a real entity created from a class, e.g. student1, student2 and student3.
- **this:** refers to the object that is currently being created or that is calling the method.

Differences between JavaScript functions and classes

Point	Function	Class
Purpose	Performs a task / returns a value; can be used as an old-style constructor.	Blueprint for creating many objects with shared properties and methods.
Syntax	function name(params) { ... }	class Name { constructor() { ... } method() { ... } }
Creating objects	Only when written as a constructor function and called with new.	Always created with new Name(...).
Calling without new	Allowed (a normal call; this may be undefined/global).	Not allowed – throws TypeError: Class constructor cannot be invoked without 'new'.
Methods	Attached separately: Fn.prototype.method = function() {...}	Written inside the class body; automatically placed on the prototype.
Hoisting	Function declarations are hoisted and can be called before they are defined.	Not usable before the declaration (temporal dead zone – ReferenceError).
Strict mode	Non-strict by default.	Class body always runs in strict mode.
Inheritance	Manual prototype linking (Object.create, call).	Simple with extends and super().
typeof	'function'	'function' (a class is a special function).
Best used for	Small reusable operations, callbacks, utilities.	Modelling real-world entities such as Student, Car, Employee.

How a class creates multiple objects

A class is written once, but `new` can be used any number of times. Each call to `new Student( . . . )` performs these steps:

- An empty object is created.
- The object is linked to `Student.prototype`, so it can access the shared method `introduce()`.
- The constructor runs with `this` pointing to the new object and stores that object's own name, age and grade.
- The finished object is returned and saved in a variable (`student1`, `student2`, `student3`).

Common properties: every object has the same set of properties (name, age, grade), but each object holds its own values, so changing one object does not affect another. **Common methods:** `introduce()` is stored only once on `Student.prototype` and is shared by all objects, which saves memory and avoids code repetition.

Flow: Class Student (blueprint) → `new Student(...)` → constructor sets own properties → object linked to shared prototype → `object.introduce()` prints the introduction

5. ALGORITHM / PROCEDURE

1. Create a project folder and open a terminal in it (Node.js must be installed).
2. Create the file `pbm5.js`.
3. Define the class `Student` using the `class` keyword.
4. Write a constructor(`name`, `age`, `grade`) that stores the three values using `this.name`, `this.age` and `this.grade`.
5. Add the method `introduce()` that returns a sentence built from the object's own properties.
6. Create three objects `student1`, `student2` and `student3` using `new Student(...)` with different values.
7. Call `introduce()` on each object and print the returned text using `console.log()`.
8. Run the program using `node pbm5.js`.
9. Execute the additional test cases (independence of objects, shared method, `instanceof`, calling without `new`, constructor-function version).
10. Verify the actual output against the expected output for each test case.

6. PROGRAM

Command used

```
node pbm5.js
```

pbm5.js

```
// Define a class
class Student {
  constructor(name, age, grade) {
    this.name = name;
    this.age = age;
    this.grade = grade;
  }

  // Method
  introduce() {
    return `Hi, I'm ${this.name}, ${this.age} years old, in grade ${this.grade}.`;
  }
}

// Create multiple objects from the class
const student1 = new Student("Alice", 14, "9th");
const student2 = new Student("Bob", 15, "10th");
const student3 = new Student("Charlie", 13, "8th");

// Use the method
console.log(student1.introduce());
console.log(student2.introduce());
console.log(student3.introduce());
```

7. CODE EXPLANATION

Code	Explanation
<code>class Student { ... }</code>	Declares a class named Student. It is the blueprint used to create student objects.
<code>constructor(name, age, grade)</code>	Special method that runs automatically each time new Student(...) is called and receives the values for the new object.
<code>this.name = name; this.age = age; this.grade = grade;</code>	this refers to the object being created. Each object gets its own copy of name, age and grade.
<code>introduce() { ... }</code>	Method defined once inside the class. It is shared by all objects through Student.prototype.
<code>return `Hi, I'm \${this.name}, ...`;</code>	Template literal that inserts the calling object's name, age and grade into the sentence and returns it.
<code>new Student("Alice", 14, "9th")</code>	The new keyword creates an empty object, links it to Student.prototype, runs the constructor and returns the object.
<code>const student1, student2, student3</code>	Three separate objects created from the same class, each with different data.
<code>student1.introduce()</code>	Calls the shared method; inside it, this is student1, so Alice's details are used.
<code>console.log(...)</code>	Prints the string returned by introduce() in the terminal.

8. TEST CASES AND EXECUTION DATA

TC-01 was executed on the student's system (screenshot in Output 1). TC-02 to TC-07 were executed with Node.js using the Student class of pbm5.js followed by the test lines shown in Section 9 (TC-07 uses its own constructor-function code). The actual result is the text printed in the terminal.

Test Case	Request / Action	Expected Result	Actual Result	Status
TC-01	node pbm5.js	Three introduction lines for Alice, Bob and Charlie	Hi, I'm Alice, 14 years old, in grade 9th. Hi, I'm Bob, 15 years old, in grade 10th. Hi, I'm Charlie, 13 years old, in grade 8th.	Pass
TC-02	Create one object and call introduce(); print the object	Alice's sentence; object shows its own name, age and grade	Hi, I'm Alice, 14 years old, in grade 9th. Student { name: 'Alice', age: 14, grade: '9th' }	Pass
TC-03	Change student1.name; compare with student2	Only student1 changes; objects are different (false)	Alicia Bob false	Pass
TC-04	Compare introduce of two objects; check own property	Same shared method (true); not an own property (false); found on Student.prototype (true)	true false true	Pass
TC-05	typeof Student, instanceof, Object.keys()	function, true, and only the three properties (not the method)	function true ['name', 'age', 'grade']	Pass
TC-06	Call Student(...) without new	TypeError – class cannot be called without new	TypeError: Class constructor Student cannot be invoked without 'new'	Pass
TC-07	Same logic using a constructor function + prototype	Same style of output; shared method (true)	Hi, I'm Alice, 14 years old, in grade 9th. Hi, I'm Bob, 15 years old, in grade 10th. true	Pass

9. OUTPUT

Output 1: TC-01 - Running pbm5.js (three objects created from the Student class)

```
PS C:\Users\varsh\OneDrive\Desktop\html> node pbm5.js
Hi, I'm Alice, 14 years old, in grade 9th.
Hi, I'm Bob, 15 years old, in grade 10th.
Hi, I'm Charlie, 13 years old, in grade 8th.
PS C:\Users\varsh\OneDrive\Desktop\html>
```

node pbm5.js created three Student objects and printed each object's introduction using the shared introduce() method.

Output 2: TC-02 - One object and its own properties

Test code (added after the Student class):

```
const student1 = new Student("Alice", 14, "9th");
console.log(student1.introduce());
console.log(student1);
```

```
$ node tc02.js
Hi, I'm Alice, 14 years old, in grade 9th.
Student { name: 'Alice', age: 14, grade: '9th' }
```

introduce() returned Alice's sentence and the object stores its own name, age and grade.

Output 3: TC-03 - Objects keep their own values

Test code (added after the Student class):

```
const student1 = new Student("Alice", 14, "9th");
const student2 = new Student("Bob", 15, "10th");
student1.name = "Alicia";
console.log(student1.name);
console.log(student2.name);
console.log(student1 === student2);
```

```
$ node tc03.js
Alicia
Bob
false
```

Changing student1.name did not change student2.name, so each object has independent property values.

Output 4: TC-04 - The method is shared by all objects

Test code (added after the Student class):

```
const student1 = new Student("Alice", 14, "9th");
const student2 = new Student("Bob", 15, "10th");
console.log(student1.introduce === student2.introduce);
console.log(student1.hasOwnProperty("introduce"));
console.log(Student.prototype.hasOwnProperty("introduce"));
```

```
$ node tc04.js
true
false
true
```

introduce is not stored in each object; both objects use the single copy on Student.prototype.

Output 5: TC-05 - typeof, instanceof and own properties

Test code (added after the Student class):

```
const student1 = new Student("Alice", 14, "9th");
console.log(typeof Student);
console.log(student1 instanceof Student);
console.log(Object.keys(student1));
```

```
$ node tc05.js
function
true
[ 'name', 'age', 'grade' ]
```

A class is a special function; the object is an instance of Student and holds only the three data properties.

Output 6: TC-06 - Calling a class without new

Test code (added after the Student class):

```
try {
  const s = Student("Dan", 12, "7th");
} catch (e) {
  console.log(e.name + ": " + e.message);
}
```

```
$ node tc06.js
TypeError: Class constructor Student cannot be invoked without 'new'
```

Unlike an ordinary function, a class cannot be called without new.

Output 7: TC-07 - Same idea using a constructor function (for comparison)

Test code (tc07.js):

```
// Same idea written with a constructor function (pre-ES6 style)
function StudentFn(name, age, grade) {
  this.name = name;
  this.age = age;
  this.grade = grade;
}
StudentFn.prototype.introduce = function () {
  return `Hi, I'm ${this.name}, ${this.age} years old, in grade ${this.grade}.`;
};
const s1 = new StudentFn("Alice", 14, "9th");
const s2 = new StudentFn("Bob", 15, "10th");
console.log(s1.introduce());
console.log(s2.introduce());
console.log(s1.introduce === s2.introduce);
```

```
$ node tc07.js
Hi, I'm Alice, 14 years old, in grade 9th.
Hi, I'm Bob, 15 years old, in grade 10th.
true
```

A constructor function with a prototype method produces the same result, but a class keeps the constructor and methods together in one block.

10. RESULT

Thus, the difference between JavaScript functions and classes was studied, and a Student class was created with a constructor and an introduce() method. Multiple objects (Alice, Bob and Charlie) were created from the same class; each object holds its own property values while all of them share the same method. The program was verified using multiple test cases.